



PDI: ATIVIDADE 03 · REGISTRO DE ENTREGA

Arquitetura **Serverless** para Processamento Assíncrono (event-driven)

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 03 · Fullstack e Modelagem de Dados

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

functions / auto-scale / pay-per-use

1. Contexto

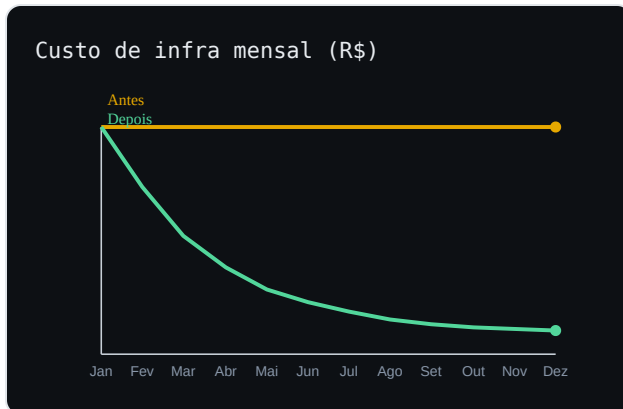
A FV roda alguns jobs de processamento em servidores dedicados 24/7, mesmo quando não há nada a fazer. Paga máquina ociosa à noite, fim de semana e feriado. Para picos, ou sobre-dimensiona (caro) ou fica sem capacidade (lento). O modelo **serverless** inverte isso: você paga só pela execução.

ANALOGIA DA LANCHAS POR HORA

Um servidor dedicado é como alugar uma lancha o mês todo: paga mesmo parada no cais. Serverless é pagar só pelos minutos em que a lancha está *efetivamente navegando*. Se ninguém usa, custo zero. Se 100 pessoas usam ao mesmo tempo, o serviço cria 100 lanchas pequenas sozinho: e some com elas quando o mar esvazia.

2. Diagnóstico

Servidores ociosos consomem orçamento sem entregar valor. Picos não previstos geram timeouts. O provisionamento manual é lento e errante: ou sobra recurso (dinheiro jogado fora) ou falta (SLA estourado).

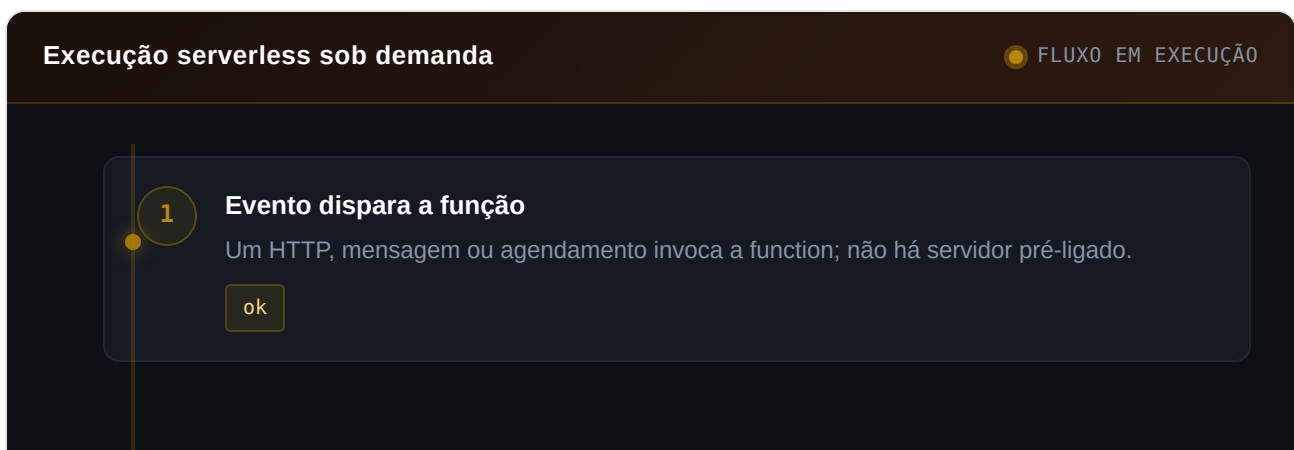


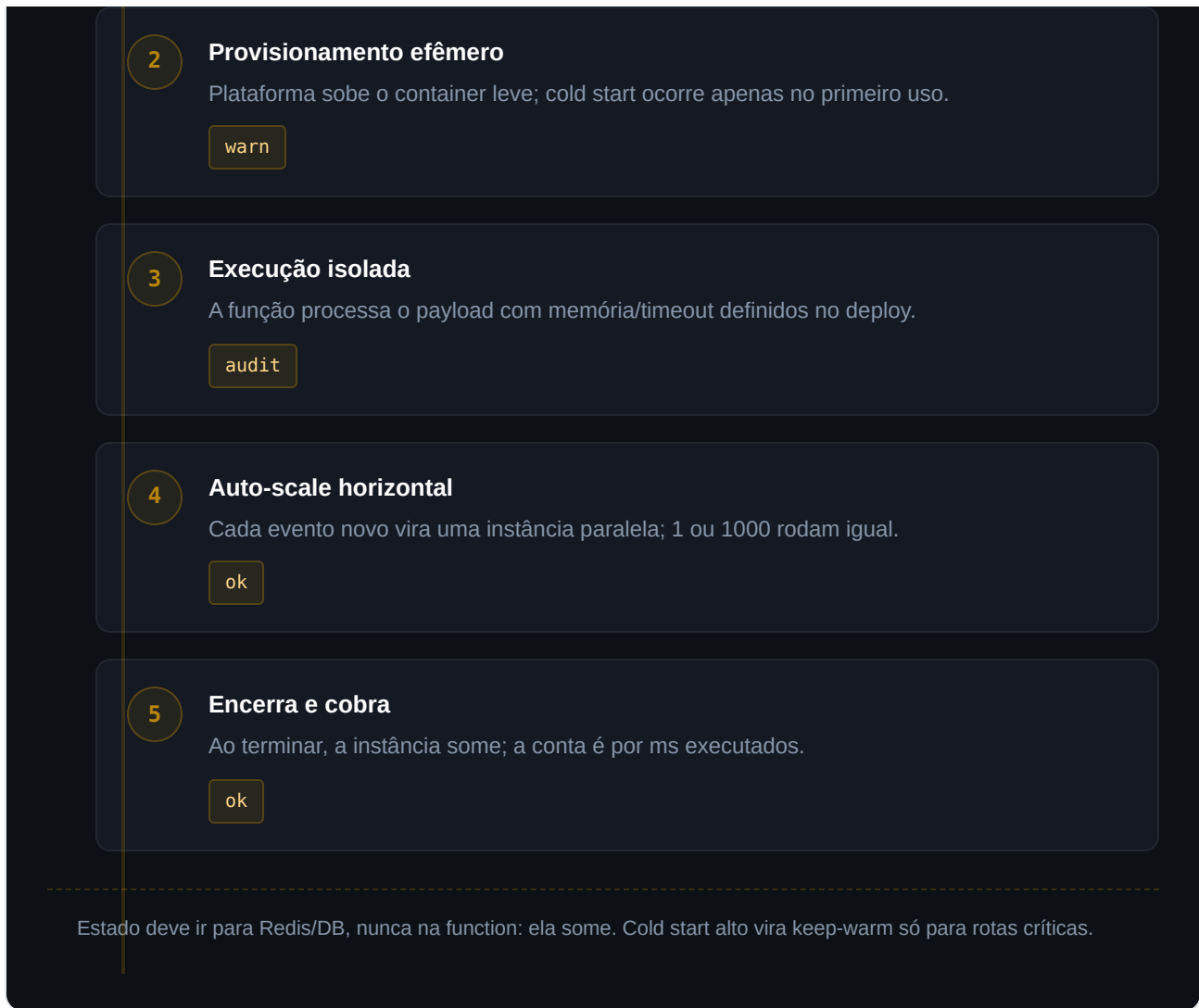
Raiz do problema: capacidade é fixa (provisionada por adivinhação), então o custo é pago mesmo sem uso e o pico quebra o teto. Serverless troca capacidade fixa por capacidade elástica e cobrança por execução.

3. Solução

Migrar jobs esporádicos e APIs de baixo/médio tráfego para **functions serverless** com **auto-scale** a partir de zero. Cobrança **pay-per-use** (por ms de execução + memória). O trade-off é o **cold start**: a primeira chamada após ociosidade leva mais tempo, mitigável com keep-warm estratégico ou tamanho de memória adequado.

4. Como funciona (pipeline)





5. Antes vs Depois

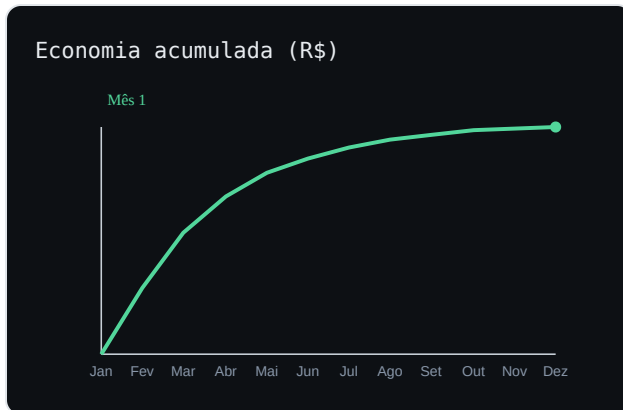
CENÁRIO	ANTES (SERVIDOR FIXO)	DEPOIS (SERVERLESS)
Custo em dia sem uso	Pago integral	R\$ 0,00
Pico não previsto	Timeout / lento	Auto-scale absorve
Provisionamento	Manual é lento	Instantâneo
Cold start	N/A	Existe, mitigado

6. Entregas

- **Function** `process_job.py`: handler sem estado, lê/escreve em Redis e DB.

- **Config** serverless.yml: memória, timeout e gatilhos (HTTP/agenda).
- **Doc** custo.md: projeção pay-per-use vs servidor dedicado.
- **Monitor** de cold start e taxa de auto-scale por função.

7. Métricas



Meta: reduzir custo de infra ociosa em 80% mantendo p95 de resposta em pico abaixo de 2,5s.

8. Decisões e tradeoffs

- **Upload nunca processa no request (grava no store e publica `file.uploaded`, consumer com concorrência limitada a 10):** request síncrono estourava timeout em planilhas de até 50k linhas. Tradeoff: resposta assíncrona exige acompanhar status por evento, aceito porque elimina o timeout.
- **Idempotência por dedup hash(arquivo + tenant):** reprocessar não duplica leads. Tradeoff: precisa de store de chaves (em prod Redis/DynamoDB; no PoC um set em memória) e custa uma leitura por evento.
- **DLQ após N tentativas + 1 arquivo ruim não afeta os outros:** isolamento por mensagem. Tradeoff: mensagens na DLQ exigem operação manual de replay; sem DLQ a fila travaria na mensagem veneno.
- **Payload carrega só metadados (URL), nunca o arquivo:** fila leve e rápida. Tradeoff: o consumer precisa buscar o objeto no store, uma chamada a mais por mensagem.
- **Quando NÃO usar (carga constante alta, worker always-on sai mais barato) + segredos fora de env hardcoded, pooler com TLS e timeout de até 60s:** serverless só onde há ociosidade (o worker ficava 90% ocioso). Tradeoff: cold start em rajada, mitigado com concorrência provisionada.

9. Impacto no negócio

Clientes enviam planilhas de 1k a 50k linhas e picos de 200 uploads derrubavam o worker always-on, 90% ocioso. Com fila, função e store, o custo cai para menos de R\$ 0,20 por mil planilhas, a escala vai a zero no vale e o p95 fica abaixo de 60s com zero duplicatas, o que viabiliza campanhas de importação em rajada sem provisionar servidor parado.

10. Referências de estudo

- Curso: AWS Lambda e Serverless na prática (Alura)
- Vídeo: Serverless em 100 segundos (Fireship, YouTube)
- Doc oficial: Cloud Run functions, cloud.google.com/functions/docs (verificada em 2026-09-28)
- Doc oficial: Terraform, developer.hashicorp.com/terraform/docs (verificada em 2026-09-28)

11. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.